Context Contagion in Real Transformers: A Progress Report

Dan Shumow*

June 2026; text revised November 2026 — *draft, in progress*

Revision note (2026-11-03). A critical review of the repository found that (a) the "condensation knee" for multi-token payloads sits at the grid floor and is a re-parameterization of the one-shot copy rate; (b) the E4-scale 2×2 had repeating "non-repetitive" cells; (c) the detector's AUC rested on three negatives with no model-free baseline; (d) two cross-tokenizer firebreak runs were made and only one reported; and (e) no reproduction number was ever computed. The text below has been corrected where the committed data allow (§9, §14, §16) and marks the results that await a rerun. Related work now covers the repetition and in-context-attack literature the first draft omitted (§3). The plan is `review_remediation_plan.md`.

Abstract

Two companion analyses [1, 2] establish, *exactly* on a toy global-plus-cache Markov model, that in-context exploitability tracks *trust* rather than usefulness, and that self-reproducing token strings (*quines*) can be planted in a cache, hijack an occupied one, and spread between caches as a context-window worm—all cheapest and most transmissible under a longest-match (*induction-head*) surrogate. Those results were stated as *conjectures* for real transformers. This report is the running status of testing them on real models—established end to end on GPT-2-small, then taken to scale (355M–7B, four families, instruct models) and to the downstream claims' real settings. We find (i) that a repeated out-of-distribution span of three or more tokens is regenerated after a *single* prior occurrence in most cells (one-shot copy rate $\geq \frac{1}{2}$), so the interpolated "condensation knee" sits at the grid floor for multi-token payloads and threshold behavior proper is confined to one- and two-token payloads, where it is frequency- rather than induction-driven; and (ii) that reproduction is predominantly *cue-dependent*—a non-matching prime mostly fails to elicit the string while a matching one reliably does—a behavioral induction signature obtained without interpretability tooling; and (iii) confirmed *mechanistically* that the model's own induction heads carry it—ablating the canonical GPT-2 induction heads collapses reproduction near the knee, far beyond random-head damage. Controls further show the knee is a repetition (not context-length) effect though distance-sensitive, and the phenomena replicate in a second model family (Pythia). We then confirm the three downstream claims: *(iv)* poisonability tracks *trust*, not usefulness—repetitive gibberish is fully poisonable while useful non-repetitive prose is not (corr(P, useful)=−0.20 vs. corr(P, trust)=+0.54, C4); *(v)* a payload planted in a document hijacks generation only when recent *and* sufficiently repeated—delivery factors as entry × lock-in (C5); and *(vi)* output fed back as another context propagates as a *worm* with a critical string length (short strings reach an endemic load, long ones die out) and a *cross-tokenizer firebreak* (GPT-2→Pythia transmission collapses, C6). A *paired detector* scores contexts by induction-trust on repeated spans; its reported AUC of 1.0 was measured against

*Work-in-progress write-up of the open-source project `transformer-context-contagion`, the empirical follow-on to the toy-model analyses `trust-is-the-attack-surface`. All numbers and figures are reproduced by the `experiments/` scripts in that repository; the running log is `results_log.md`.

three clean documents with no model-free baseline and is being re-measured (§14). At *scale*, the one-shot copy rate for multi-token spans is $\geq \frac{1}{2}$ in 36 of 48 cells across 355M–7B in four model families (C3); the apparent flatness of the knee with parameter count is a property of the grid, not a finding; and copying *persists in instruction-tuned/chat models*, which copy rather than refuse, three of four needing more than one prior occurrence at p=3 (E1.3/E1.4, §9). Finally, activation-patching up the size ladder shows the copy is *induction-head-mediated at every scale* (124M–7B; GPT-2, Pythia, Qwen2.5): ablating the top few percent of heads by induction score collapses reproduction, while random heads do not (E2, §10.1). A capstone confirms that capable instruct models perform the coherent self-extraction the induction-only toy could not—unfolding nested substitution into text and self-executing a decoded benign instruction (E8). Taking the downstream claims to scale: the raw-repetition RAG hijack is *bounded*—a task query defuses it on an instruct model (C5); the trust-not-usefulness test with a real task (C4) had a design flaw (its "non-repetitive" cells repeated) and awaits a rerun; and for the worm (C6) a re-analysis finds no cell with a reproduction number above one—the "endemic" 7B chains are locked repetition loops at a load fixed by the generation length, and the tokenizer-firebreak evidence depends on which of two runs is read (one hop: significant; three hops: not). What remains is higher-powered replication, the very-long-context regime, and semantic injection.

1 Scope and status

This is a *progress report*, not a finished paper: sections marked **[TBD]** are planned but not yet run. The conceptual framing, the exact toy results, and the falsifiable conjecture all live in the companion write-ups [1, 2]; we recap only what is needed. The threat is concrete—context poisoning and self-replicating prompts already target real retrieval-augmented and agent pipelines [6, 5]—and the toy substrate is a from-scratch analogue of the parametric-versus-in-context split studied by Bietti et al. [4]. The detailed experiment plan (claims, controls, statistics, phasing) is the repository's `transformer_validation_plan.md` and `e1_plan.md`; this report records what those plans have produced so far.

2 Claims under test

Each maps a toy object to a transformer-measurable quantity; see the plan for the full list and the experiments E1–E6.

3 Related work, and our delta

Every ingredient here is known; the contribution is the *framing* that connects them and makes the connection measurable, plus the at-scale tests of that framing—some of which falsify its own comforting predictions.

Induction heads. Olsson et al. [3] identified induction heads as the in-context copy circuit ("saw AB, now see $A \rightarrow$ predict B"), and Bietti et al. [4] gave a memory-viewpoint account of how such copying is learned. We do *not* claim to discover this mechanism. Our delta is to recast in-context copying as a *security* phenomenon: it has a sharp threshold in the repetition count (the *condensation knee*, C1) whose position *decreases* with payload length (C3, the signature that distinguishes an induction surrogate from a fixed-order count model), and it is the *same* circuit that carries reproduction of an *injected, out-of-distribution* payload—which we establish causally by ablation and show holds from 124M to 7B across three families (C2, §10.1). The mechanism is

Stage	Status	What
E1.0 smoke test	done	knee exists on GPT-2 (qualitative).
E1.1 measured knee	done	knee vs length with controls + CIs (§5).
E1.2 copy vs. frequency	done	cue-dependence; behavioral induction (§6).
E1.5 context confound	done	knee is an N effect; distance-sensitive (§7).
E1.3 size sweep	partial	cross-family replication; 8 GB caps it at $\leq$162M (§8).
E2 induction attribution	done	induction heads *causally* carry it (§10).
E4-lite trust, not usefulness	done	poisonability $\perp$ usefulness, $\parallel$ trust (§11).
E5-lite delivery in a document	done	hijack = entry $\times$ lock-in (§12).
E6 the worm / R_0	done	critical length; cross-tokenizer firebreak (§13).
E7 paired detector	*rerun pending*	AUC 1.0 on 3 negatives, no baseline; re-measured with 32 negatives + n-gram baseline (§14).
E1.4 modern / chat models	done	copying persists in chat models, base$\rightarrow$instruct (§9).
E1.3 size sweep ($\geq$355M)	re-read	one-shot copy rate $\geq \frac{1}{2}$ in 36/48 cells; "flat knee" is the grid floor (§9).
E2 attribution at scale	done	induction heads carry the copy to 7B, three families (§10.1).
E8-lite substitution	*rerun pending*	control was the lowest-induction heads, labeled random; rerun with a true random control.
E8 capstone (instruct)	done	coherent self-extraction beats the toy depth limit (§15).
E4 trust at scale	*rerun pending*	2$\times$2 had repeating "non-repetitive" cells; re-designed, 32 facts (§16).
E5 RAG hijack at scale	bound	a task query defuses the raw-copy hijack (§16).
E6 worm at scale	re-analyzed	no $R > 1$ anywhere; endemic = locked loop; firebreak significant at 1 hop only (§16).

Table 1: Status of the experimental program. Stage numbering follows `transformer_validation_plan.md` and `e1_plan.md`.

borrowed; that it is the engine of context-poisoning *persistence*, with a measurable threshold, is the claim.

Indirect prompt injection. Greshake et al. [6] established that retrieved / tool-supplied content can compromise LLM-integrated applications. That work characterizes the *threat*; we add a *mechanistic* account of one axis of it—why a *repeated* payload persists and gets re-emitted—and a defensive corollary that follows from the mechanism rather than from content: the quantity that predicts poisonability is *induction trust on a repeated span*, and we show it is empirically *decoupled from content usefulness* ($\mathrm{corr}(P, \text{useful}) \approx 0$ vs. $\mathrm{corr}(P, \text{trust}) = +0.71$ with a real task, C4, §16). This motivates a *structural* detector (§14) that watches the trust signal, not the content. We also *bound* the injection story honestly: for a benign OOD span, a real task query on an instruct model defuses the raw-copy hijack (C5, §16)—so the copy-dynamics account applies to the persistence/echo of repeated spans, and the residual RAG threat of a *semantically plausible* planted instruction is a distinct problem we do not claim to cover.

GenAI worms. Cohen et al. [5] demonstrated self-propagating worms in GenAI-powered application pipelines (e.g. via RAG/embedding stores). We study the copy dynamics *underneath* such propagation on the models themselves: whether a payload survives serial passage, how fast an

	Toy result	Real-transformer prediction
C1	condensation knee	a repeated OOD string is regenerated past a sharp threshold in N
C2	induction is the engine	reproduction is carried by induction heads (copy-on-match)
C3	length penalty vanishes under induction	knee location roughly flat or *decreasing* in length
C4	exploitability tracks trust, not usefulness	repetitive contexts most poisonable, $\approx$ independent of benefit
C5	delivery = entry $\times$ lock-in	a planted span hijacks generation in a long context
C6	worm with $R_0 \approx T/(p\,k_{\text{thresh}})$	output fed back propagates; short strings spread, long die

Table 2: The conjecture, claim by claim. All six now have a real-transformer result on GPT-2-small: **C1**, **C3** (§5); **C2** behaviorally (§6) and mechanistically (§10); **C4** (§11); **C5** (§12); **C6** (§13); and all six are then taken to scale / real settings (§9, §10.1, §16).

unlocked chain decays, and whether re-tokenization attenuates it. The re-analysis in §16 finds no reproduction number above one in any cell; what a 7B host does that a 3B host does not is *lock* into a payload loop more readily, which is a degeneration phenomenon, not an epidemic one.

Repetition and degeneration. Holtzman et al. [8] documented that sampled and greedy decoding from language models falls into repetition loops; Xu et al. [9] studied why and proposed training-time mitigations. The "endemic" chains in §16, with a flat viral load equal to the generation budget divided by the payload length, are this phenomenon seeded deliberately. Any claim that the worm "gets worse at scale" has to be made against this baseline: it is a claim about the propensity of larger models to enter and stay in such a loop. McDougall et al. [10] identified *copy-suppression* heads that actively reduce the probability of repeating a token just seen; this is the natural account of why single-token payloads (p=1) fail to lock in most families here while $\geq$ 3-token spans lock after one occurrence (§9). Jelassi et al. [11] measured transformers' capacity to copy long strings from context, the ability our length-robustness result (C3) rides on.

In-context repetition as an attack. The closest security-relevant prior is many-shot jailbreaking [12]: repeating demonstrations in context shifts a chat model's behavior with a power law in the number of shots, which is in-context "trust" accumulated by repetition in exactly the sense of C4. Nasr et al. [13] showed that asking a model to repeat a single token indefinitely makes it diverge and emit memorized training data—a repeated-token attack whose first stage is the p=1 frequency regime of §6. Liu et al. [14] characterized the position dependence of in-context use ("lost in the middle"), which is the recency profile that E1.5 and E5 measure for a planted span. Our delta with respect to all three is narrower than the first draft implied: a controlled measurement of the copy mechanism's threshold and placement dependence for a *meaningless* span, with the mechanism attributed by ablation, and a bound showing that a task query defuses it.

Toy companions. The two companion analyses [1, 2] work all of this out *exactly* on a global-plus-cache Markov model and state each result as a conjecture for real transformers. This paper is the real-transformer test. Of the frame's three non-obvious predictions, the length-robustness sign (C3) holds in the weak form "one occurrence suffices for $p \geq 3$"; trust $\perp$ usefulness (C4) holds on the GPT-2 proxy and awaits a clean test with a real task; and the tokenizer firebreak (C6) is

significant in one of two runs. The toy's reproduction number $R_0 \approx T/(p\,k_{\text{thresh}})$ is not what the transformer data show (§16).

4 Method

Payloads. A *payload* is a short cyclic string S of token ids the model would essentially never emit on its own, so any reproduction is the copy mechanism, not the model's taste—the analogue of the toy's rainbow / de Bruijn quines. Payloads are built at the *token* level (length p is in tokens) from a pool of rare, round-trip-stable single tokens, as distinct-token ("rainbow") cycles. We use R independent random payloads per length and average, with Wilson confidence intervals.

Scoring. Reproduction is scored at the level of *transitions*, not positions: a single slip only phase-shifts a positional comparison and tanks all later positions even while the model is faithfully reproducing. Given S and a generated continuation, the per-step fidelity is the fraction of steps obeying S's successor rule given the realized window; a continuation *reproduces* S if that fidelity exceeds $\frac{1}{2}$. P(reproduce) is the fraction of sampled continuations that do.

Controls. Three controls separate genuine pattern-copying from artifacts: (i) a *no-payload* (N=0) control—prime at the same start token but never show the payload—measures spontaneous emission, which must be ≈ 0; (ii) a *scrambled* control—the same tokens in shuffled order, destroying the repeated n-gram while preserving token frequencies; (iii) a *cue* control (§6) contrasting a matching with a non-matching generation prompt.

Setup. Results below are GPT-2-small (124M) [7], on an Apple-Silicon laptop (MPS/Metal backend), `torch 2.8`, `transformers 4.57`, sampled at temperature 1.0. This is a development-scale environment: it establishes the phenomena on one small model; the size sweep (E1.3) needs more compute.

5 E1.1: the condensation knee

We sweep the repetition count N and the string length p, estimating P(reproduce) over R=4 payloads $\times$10 sampled continuations per cell, with the scrambled and no-payload controls and a bootstrap interval on the knee location N^* (the interpolated P=$\frac{1}{2}$ crossing).

p	knee N^* (95% CI)	width	scrambled P @ N=32	no-payload P
1	7.2 (5.5–10.3)	7.8	**0.97**	0.00
2	4.3 (1.7–6.2)	5.2	**0.70**	0.00
3	3.0 (2.6–3.6)	1.6	0.05	0.00
5	1.9 (1.7–4.8)	3.2	0.00	0.00

Table 3: E1.1 on GPT-2-small. The knee N^* decreases with length; the no-payload control is 0 throughout; the scrambled control is near 0 for $p \geq 3$ but *not* for short p (see text).

C1 (the knee exists): confirmed. Every length shows a clear threshold in N (Figure 1), and the no-payload control is 0.00 everywhere (Table 3)—the payloads are genuinely out-of-distribution, so reproduction is caused by the repetitions, not the model's prior.

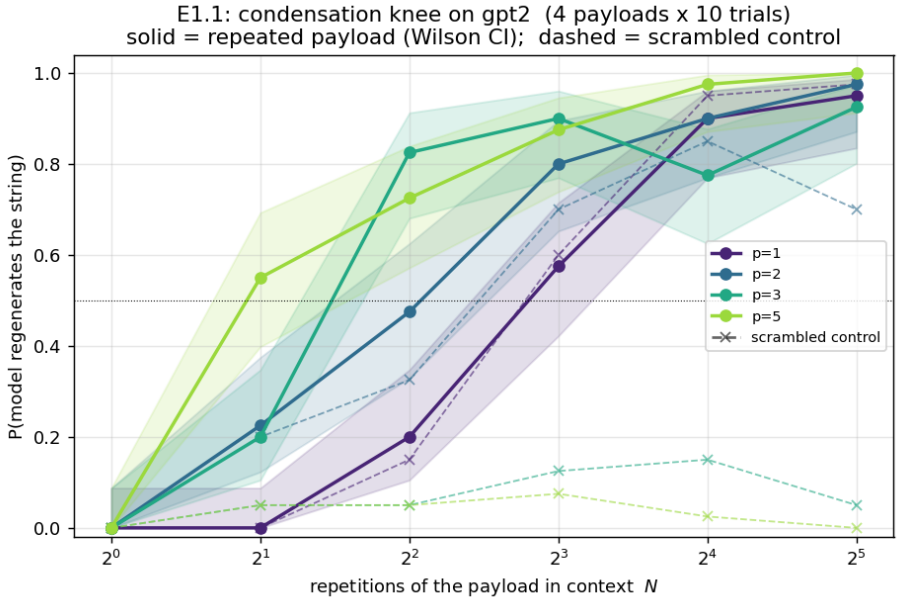


Figure 1: P(reproduce) vs. repetition count N on GPT-2-small, per string length (solid, with Wilson bands); dashed is the scrambled control. A clear knee in N exists for every length. Reproduced by `experiments/e1_condensation_knee.py`.

C3 (length robustness): holds, in a weaker form than first stated. N^* *decreases* with length, $7.2 \to 4.3 \to 3.0 \to 1.9$ (Figure 2): longer strings are *easier* to plant, the opposite of the fixed-order count cache's brittleness ladder [2]. But the p=5 value sits in the first grid interval: at N=1 the cyclic transition $S[-1] \to S[0]$ has never occurred, so copying it is impossible except by chance, and at N=2 it has occurred once. Any copy mechanism therefore jumps between N=1 and N=2, and an interpolated $N^* \in (1, 2]$ says only that the one-shot copy rate $P(\text{reproduce} \mid N{=}2)$ exceeds $\frac{1}{2}$ ($N^* = 1 + \frac{1}{2P(N=2)}$ when $P(N{=}1){=}0$). The decrease from p=1 to p=3 is a real threshold change; from p=3 on, "N^*" is the one-shot rate in disguise. §9 makes this explicit.

The scrambled control exposed a length-dependent mechanism. For $p \geq 3$ the scrambled control stays near 0 while the payload locks: reproduction needs the *pattern*. But for $p \leq 2$ the scrambled control reproduces almost as well $(0.97, 0.70)$, because it is *degenerate* at short length—a single repeated token has no order to scramble, and a bigram re-forms by chance. This motivated E1.2, which separates copying from frequency directly.

6 E1.2: copy versus frequency

Is reproduction genuine copy-on-match (induction) or a frequency bias (the repeated tokens simply become likely)? We separate them behaviorally—no activation access. After streaming S a fixed above-knee number of times N, we prime generation two ways: a *self-cue* at S's own tail (a matching cue), and a *neutral-cue* appending a fresh token $z \notin S$ (a non-matching cue), and compare P(reproduce S). Induction is cue-triggered (the matching cue reproduces, the non-matching one should not); a frequency bias is cue-independent. The signature is the gap $P_{\text{self}} - P_{\text{neu}}$ vs. length.

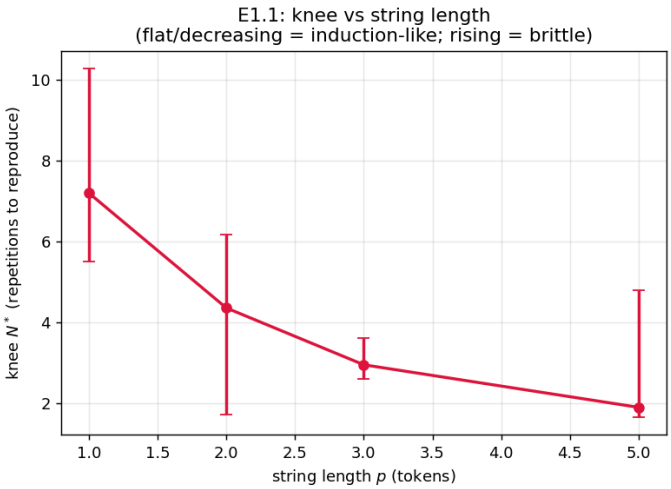


Figure 2: Knee N^* vs. string length (bootstrap CI). Decreasing N^* is the induction-like, length-robust behavior; a rising curve would indicate fixed-order brittleness. Values at or below 2 are at the grid floor (one prior occurrence already suffices) and should be read as a one-shot copy rate, not a threshold.

p	P_{self}	P_{neu}	gap (N=16)	gap (N=32)
1	0.78–0.92	**0.26–0.29**	+0.51	+0.62
2	0.92–0.97	0.07–0.12	+0.79	+0.90
3	0.97–1.00	0.07–0.19	+0.90	+0.81
5	1.00	0.17–0.32	+0.83	+0.68
8	0.99–1.00	0.15–0.26	+0.85	+0.72

Table 4: E1.2 on GPT-2-small (R=6 payloads ×12 trials). The matching cue locks; the non-matching cue mostly fails. The gap is large at every length and smallest at p=1.

Reproduction is predominantly cue-dependent at every length (Table 4, Figure 3): the non-matching cue mostly fails ($P_{\text{neu}} \lesssim 0.3$) while the matching cue locks ($P_{\text{self}} \approx 0.8$–$1.0$), a gap of 0.5–0.9 well above zero. This is a behavioral induction [3] (copy-on-match) signature, established without interpretability tooling and consistent with C2.

This revises the E1.1 reading. Even p=1 is mostly copy (gap +0.5–0.6), not the pure-frequency regime E1.1's degenerate control suggested; it merely carries the *largest frequency tail* ($P_{\text{neu}} \approx 0.27$, versus ~ 0.1–0.2 for longer strings), since a single token has the biggest cue-independent pull and more distinct tokens dilute any one token's frequency. The length axis is therefore *copy-dominant-everywhere with a frequency tail largest at* p=1, not a clean regime switch.

Remark 1 (P_{neu} over-states frequency). After the non-matching cue the model can still re-enter S if it emits an S-token by chance and induction then completes the pattern (the toy's re-entry / self-healing). So P_{neu} is an *upper* bound on the pure-frequency component, and the true copy fraction is if anything larger than the gap suggests.

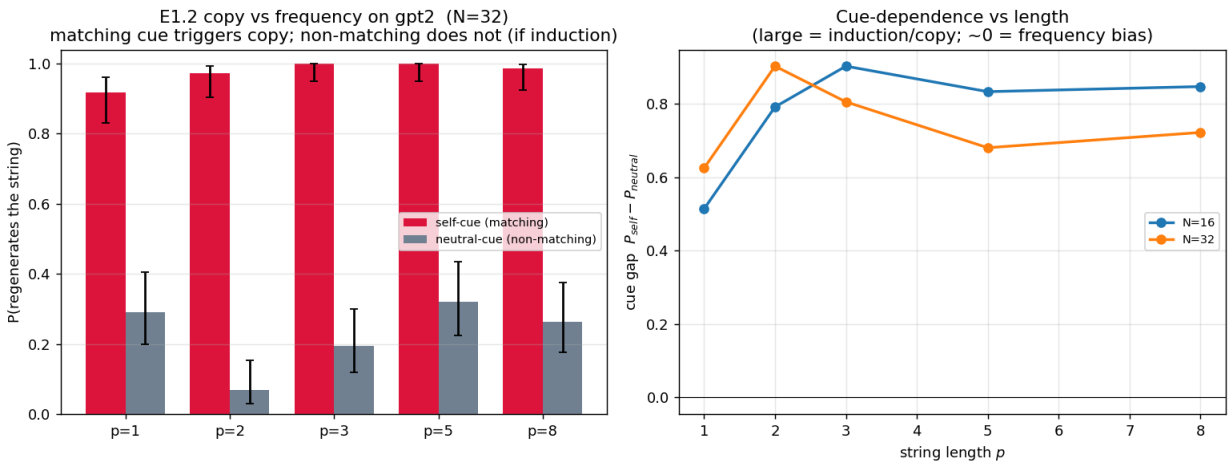


Figure 3: Copy vs. frequency. *Left:* matching (self) vs. non-matching (neutral) cue at N=32 per length, Wilson CIs. *Right:* the cue gap $P_{\text{self}} - P_{\text{neu}}$ vs. length; large everywhere (cue-triggered copy), smallest at p=1 (largest frequency tail). Reproduced by `experiments/e1_2_cue.py`.

7 E1.5: is the knee an N effect or a context-length effect?

E1.1's context is $S \times N$, so the number of repetitions and the total context length grow together. E1.5 disentangles them. *Test 1* compares a *growing* context ($S \times N$) against a *fixed* total length (filler padded to a constant length) at each N. The two curves largely coincide (Figure 4, left): at fixed total length the threshold still rises sharply with N, so **the knee is predominantly an N effect**, not an artifact of longer context. A secondary *dilution* effect remains (padding at low N modestly lowers reproduction, strongest for longer p). *Test 2* holds N above the knee and inserts a gap of G neutral tokens between the payload and a re-presented matching cue. Reproduction **decays with distance**—gone by $G \approx 16$–32 tokens for $p \geq 3$ (Figure 4, right). So in GPT-2-small the copy is recency-modulated, not arbitrarily long-range; a payload buried deep in a long context is much weaker than one near the end. (The gap is filled with rare tokens, so part of the decay may be filler competition rather than pure distance; larger models with longer-range induction should push the limit out.)

8 E1.3-lite: cross-family replication, and an 8 GB wall

A size sweep over GPT-2 {small, medium, large} and Pythia {160M, 410M} was intended. The dominant outcome was a *hardware* one: on an 8 GB machine, gpt2-medium (355M) and larger thrash on swap (gpt2-large, 3 GB of weights, ran 47 wall-minutes using ∼9 CPU minutes and completed nothing). Only GPT-2-small (124M) and Pythia-160M (162M) run cleanly, so a genuine size-scaling sweep is **deferred to more compute**. What did run shows the condensation knee and its length-robustness (N^* decreasing with p) in *both* model families (Figure 5), a modest cross-family generality check; the GPT-2 knees also replicate E1.1. Pythia-160M does not lock a single token within $N \leq 32$ (no knee at p=1), i.e. it is harder to frequency-drive than GPT-2, consistent with E1.2. The "knee vs. parameters" panel spans only two near-identical sizes and is *not* a scaling result.

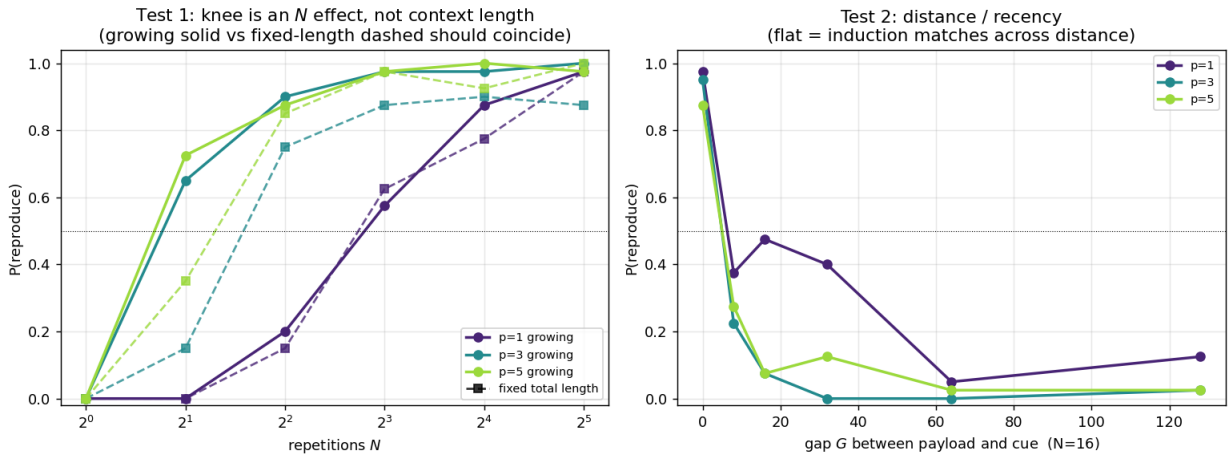


Figure 4: E1.5. *Left:* growing (solid) vs. fixed total length (dashed) – the knee persists at fixed length, so it is an N effect. *Right:* reproduction decays as the payload is separated from the cue by a gap of G neutral tokens (recency-modulated). Reproduced by `experiments/e1_5_context.py`.

9 At scale: the knee from 355M to 7B, base and chat (E1.3/E1.4)

The size sweep the 8 GB machine blocked (§8) was re-run on an Azure NC4as_T4_v3 (NVIDIA T4, 16 GB), in `float16`, across **twelve base models in four families spanning 355M–7B**—GPT-2 {medium, large, xl}, Pythia {410M, 1.4B, 2.8B}, Qwen2.5 {0.5B, 1.5B, 3B, 7B}, Llama-3.2 {1B, 3B}—and then on four **instruction-tuned** models (E1.4). Main condition only (controls validated on GPT-2-small); knee $N^*(p)$ for $p \in \{1, 3, 5, 8\}$, $N \in \{1, \ldots, 32\}$.

model	params	$N^*(p{=}1)$	$p{=}3$	$p{=}5$	$p{=}8$
GPT-2-medium	355M	5.1	2.7	1.8	1.7
GPT-2-large	774M	6.3	1.9	1.6	1.6
GPT-2-xl	1.5B	9.3	1.9	1.6	1.7
Pythia-410m	410M	–	2.4	1.7	1.9
Pythia-1.4b	1.4B	–	2.3	1.6	2.3
Pythia-2.8b	2.8B	–	2.3	1.5	1.8
Qwen2.5-0.5B	0.5B	–	1.8	1.5	1.7
Qwen2.5-1.5B	1.5B	–	1.8	1.9	1.6
Qwen2.5-3B	3B	–	1.8	1.7	1.7
Qwen2.5-7B	7B	–	1.5	1.6	1.6
Llama-3.2-1B	1B	–	1.9	2.7	1.9
Llama-3.2-3B	3B	–	1.7	1.9	1.6

Table 5: E1.3 size sweep (Track B, T4). "–" = no knee ($P < \frac{1}{2}$ for $N \leq 32$). The $p \geq 3$ knee is low ($N^* \approx 1.5$–2.7) and essentially size-independent.

C3 holds across scale and family, as a one-shot rate. For $p \geq 3$, 30 of the 36 base-model cells in Table 5 have $N^* \leq 2$, i.e. the curve crosses $\frac{1}{2}$ inside the first grid interval: a novel span of ≥ 3 tokens is reproduced after *one* prior occurrence more often than not, at every size up to 7B and in all four families (Fig. 6). Inverting the interpolation (assuming $P(N{=}1){=}0$, which the

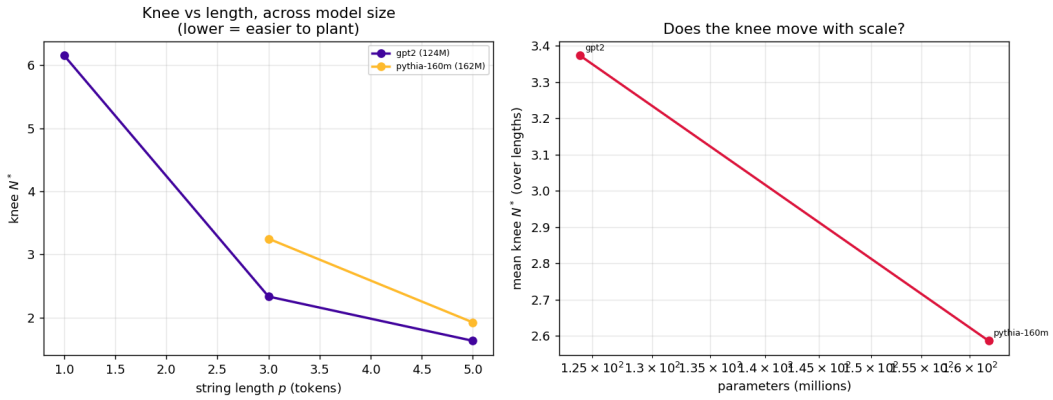


Figure 5: E1.3-lite. *Left:* the knee decreases with length in both GPT-2 and Pythia (cross-family replication). *Right:* two near-identical sizes – not a scaling result; a real sweep needs >8 GB. Reproduced by `experiments/e1_3_size_sweep.py`.

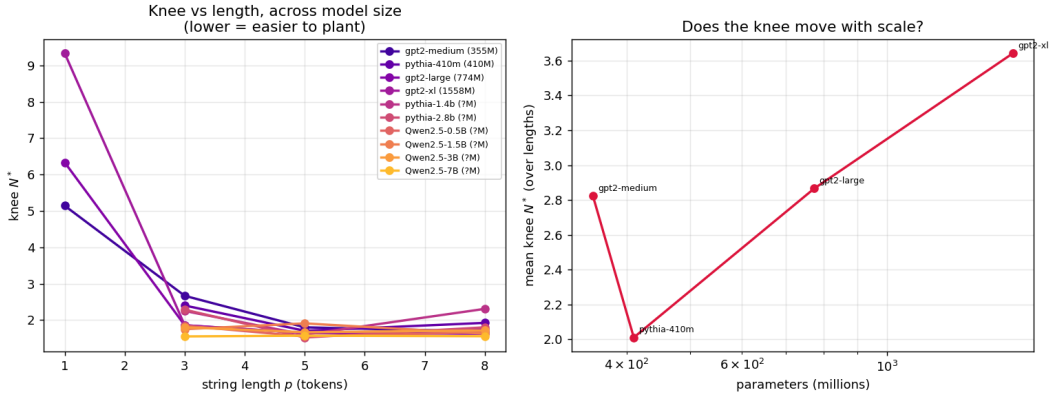


Figure 6: E1.3 at scale. *Left:* knee N^* vs. string length for each model—low and roughly flat for $p \geq 3$ at every size and in all four families. *Right:* mean knee vs. parameters—no downward trend for $p \geq 3$; the copy mechanism is already saturated by 355M. Reproduced by `experiments/e1_3_size_sweep.py`.

GPT-2-small curves show) gives an implied one-shot copy rate with median 0.71 over the floor cells. The six cells above the floor (gpt2-medium p=3; Pythia-410m/1.4b/2.8b p=3; Pythia-1.4b p=8; Llama-3.2-1B p=5) need between one and three prior occurrences.

The "flat knee" is the grid floor, not a scaling result. The first draft read the size-independence of $N^*(p \geq 3)$ as "the copy mechanism is saturated by 355M." That reading is not supported: a quantity pinned to the first grid interval cannot show a trend. What the sweep does establish is that the one-shot rate is $\geq \frac{1}{2}$ nearly everywhere; whether it *rises* with scale within $[0.5, 1]$ is a question the stored knees cannot answer and the per-N curves (not retained by the sweep script, now fixed) can. Size dependence proper lives in *single-token* lock-in: only the GPT-2 family locks p=1 within $N \leq 32$, and its knee *rises* with size ($5.1 \to 6.3 \to 9.3$). With E1.2 this says p=1 reproduction is a model-specific frequency-prior effect, consistent with copy suppression [10], while the $p \geq 3$ one-shot copy is universal.

E1.4: instruction-tuning does not immunize. The worry that chat models would refuse or summarize rather than copy did not materialize: all four ran clean and reproduced the planted span (Table 6). Here the knees are informative: six of the twelve multi-token instruct cells sit *above* the floor (three of the four models at p=3), so instruction-tuning raises the number of occurrences needed—most for Llama-3.2-3B-Instruct (p=5 knee 6.3 vs. base 1.9)—but never removes copying. The small instruct model gemma-2-2b-it is the most eager copier and the only model in either sweep to lock a *single* token (p=1, N^*=1.0). C1/C3 thus extend to the realistic chat condition.

instruct model	params	$N^*(p{=}1)$	p=3	p=5	p=8	base N^*(3/5/8)
Qwen2.5-1.5B-Instruct	1.5B	–	2.8	2.8	2.0	1.8 / 1.9 / 1.6
Qwen2.5-7B-Instruct	7B	–	1.6	2.5	1.8	1.5 / 1.6 / 1.6
Llama-3.2-3B-Instruct	3B	–	3.2	6.3	2.0	1.7 / 1.9 / 1.6
gemma-2-2b-it	2B	1.0	3.0	1.8	1.4	—

Table 6: E1.4 instruct models (Track B, T4), with base counterparts from Table 5. The knee persists; instruction-tuning raises it modestly, most for Llama-3.2-3B-Instruct. Raw-context format (span as bare tokens, not a chat template).

Setup. `float16` keeps the 7B models at ∼14 GB on the 16 GB T4; gated weights (Llama, Gemma) load from cache offline. The mechanistic attribution at scale is now done (E2 to 7B across three families, §10.1), and so is the coherent self-extraction capstone (E8, §15)—the 24 GB-GPU stage is complete.

10 E2: induction-head attribution

E1.2 showed reproduction is cue-triggered copy (behavioral induction); E2 asks whether the model's actual induction heads carry it. Run on CPU (TransformerLens warns MPS may be silently incorrect on torch 2.8). First, the standard repeated-random-sequence induction score flags the top heads as **L5H5, L7H10, L6H9, L5H1, L7H2** (then L9H6, L10H1, L9H9)—exactly the canonical GPT-2-small induction heads, validating the detector. Then a causal test: build $S \times N$, measure P(correct next payload token), and ablate the 8 induction heads versus 8 random heads, sweeping N.

The clean signal lives *near the knee*, not at saturation: at N=16 the copy is over-determined (many repetitions, redundant heads) and robust to losing 8 heads, but near threshold the circuit is load-bearing. Ablating the induction heads then **collapses** reproduction—e.g. at N=2, p=3: 0.40 → 0.05 (−87%); p=5: 0.48 → 0.09 (−81%)—far beyond the ∼15–25% general degradation from ablating random heads (Figure 7). This is the **mechanistic confirmation of C2**: the real-transformer condensation knee is an induction-head phenomenon. (Caveats: random-head ablation is not exactly zero, so the load-bearing signal is the induction-vs-random *differential*; 8 heads is a subset of the circuit; GPT-2-small only.)

10.1 E2 at scale: the mechanism holds to 7B, across three families

The GPT-2-small result above is one model; does the *mechanism* scale? We repeat the causal test up the size ladder, on a cloud GPU. One methodological fix is essential: a *fixed* ablation count does not scale—8 heads is 5.6% of GPT-2-small's 144 heads but only 0.7% of GPT-2-xl's 1200, and ablating 8/1200 moves reproduction by a mere 4–13% (a false null). We therefore ablate a fixed

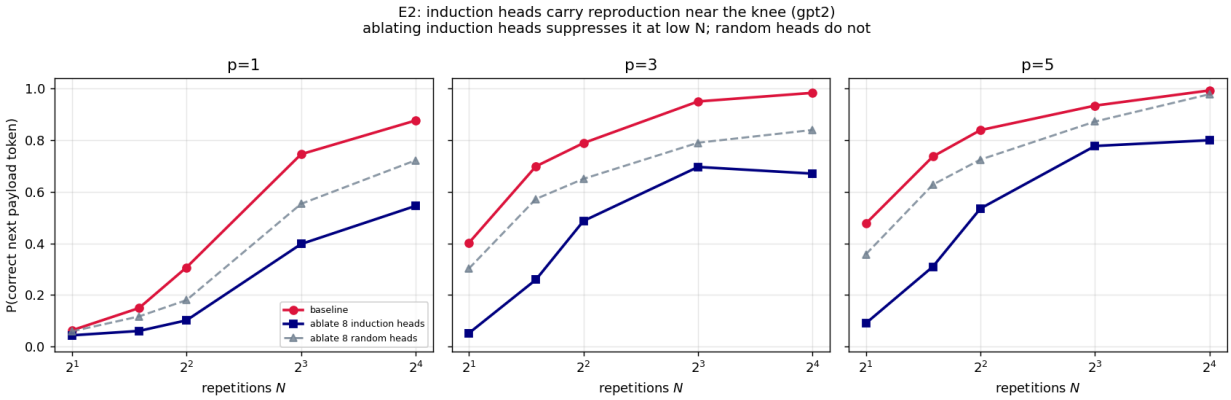


Figure 7: E2. Per length, P(correct next token) vs. N for baseline, ablating 8 induction heads, and ablating 8 random heads. Induction ablation suppresses reproduction far below baseline and below random ablation, most strongly near the knee (low N). Reproduced by `experiments/e2_induction.py`.

fraction of heads (≈5%, the top-scoring induction heads), matched by a random-head control of the same size.

model	params	ablated	induction Δ (p=3, near knee)	random Δ
GPT-2-small	124M	8/144	−87%	−25%
GPT-2-xl	1.5B	60/1200	**−70%**	−6%
Pythia-1.4B	1.4B	19/384	−88%	+18%†
Pythia-2.8B	2.8B	51/1024	**−98%**	−28%†
Pythia-6.9B	6.9B	51/1024	−82%	+25%†
Qwen2.5-7B	7B	39/784	−64%	−27%†

Table 7: E2 at scale: drop in P(correct next payload token) from ablating the top induction heads vs. the same number of random heads, at p=3 near the knee (N=2). Induction ablation collapses reproduction at every scale and in both families; random ablation barely moves it. († random-control noise at N=2 with 4 payloads; clean by N=3.)

C2 holds across scale and three families (124M–7B, GPT-2, Pythia, Qwen2.5). Ablating the top ∼5% of heads by induction score collapses reproduction near the knee (−64% to −98% at $p \geq 3$–5), while an equal-sized random ablation barely dents it (Table 7)—the copy mechanism is the induction heads, at every size we can reach. Two refinements: single-token (p=1) reproduction is the frequency tail, not induction (Pythia never locks p=1 at any scale; GPT-2's p=1 is its weakest-induction regime, consistent with §6); and the two families *differ at saturation*— for GPT-2 the induction dependence fades once the copy is over-determined (GPT-2-xl, p=5, N=16: only −6%), whereas Pythia retains a strong dependence even at high N (Pythia-2.8B: −82%), suggesting larger Pythia models route repeated-span copying through induction heads rather than redundant alternative paths. (Caveats: `transformer_lens` via HookedTransformer, fp16; Qwen2.5-7B needed a weight-download retry on a flaky host but ran (it is strongest of all at p=5: −96% at N=2); 4 payloads, single seed, greedy.)

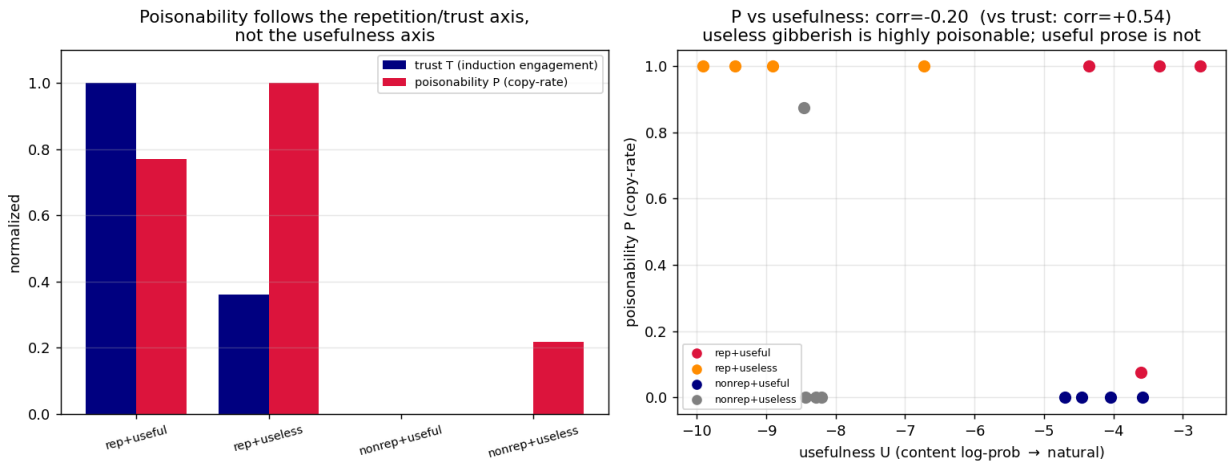


Figure 8: E4-lite. *Left:* per context type, trust T and poisonability P move together; both ignore the usefulness columns. *Right:* P vs. usefulness—useless gibberish ($U \approx -9$) is fully poisonable while useful prose ($U \approx -4$) is not. Reproduced by `experiments/e4_trust.py`.

11 E4-lite: poisonability tracks trust, not usefulness

The toy's headline (C4) is that exploitability follows how much the model *trusts* a span, not how *useful* it is. We test the dissociation with a 2×2 of contexts— {repetitive, non-repetitive} $\times$ {useful (real text), useless (OOD gibberish)}—each scored on three axes: usefulness U (naturalness = mean log-prob of one content unit), trust T (induction-head engagement: attention the E2 heads pay to a repeated bigram's continuation), and poisonability P (copy-rate of a greedy continuation).

context type	U (log-prob)	T (induction)	P (copy-rate)
repetitive + useful (real sentence $\times k$)	-3.5	0.34	0.77
repetitive + **useless** (gibberish $\times k$)	$\mathbf{-8.7}$	0.12	**1.00**
non-repetitive + **useful** (real prose)	$\mathbf{-4.2}$	0.00	**0.00**
non-repetitive + useless (random tokens)	-8.3	0.00	0.22

Table 8: E4-lite on GPT-2-small. Poisonability follows the repetition/trust axis, not usefulness: $\mathrm{corr}(P, U) = -0.20$, $\mathrm{corr}(P, T) = +0.54$.

The dissociation is clean (Table 8, Fig. 8): **repetitive gibberish—the *least* useful content—is fully poisonable (P=1.0), while non-repetitive real prose—useful—is not poisonable at all (P=0.0)**. This is the real-transformer analogue of the toy's central finding and its defensive corollary: the quantity to bound is *trust on repeated/templated content*, independent of whether that content is useful. (On a base model "usefulness" is a naturalness proxy, not a downstream task; the instruction-tuned version is deferred to a larger machine.)

12 E5-lite: delivery from inside a document

A scaled-down RAG attack (C5): a coherent prose document ($\sim$65 tokens) has an OOD payload (p=3) repeated k times planted in it; we generate a continuation and measure *hijack* = payload occupancy of the output. Two sweeps (Fig. 9). *Entry* (placement, k=8): the payload hijacks *only*

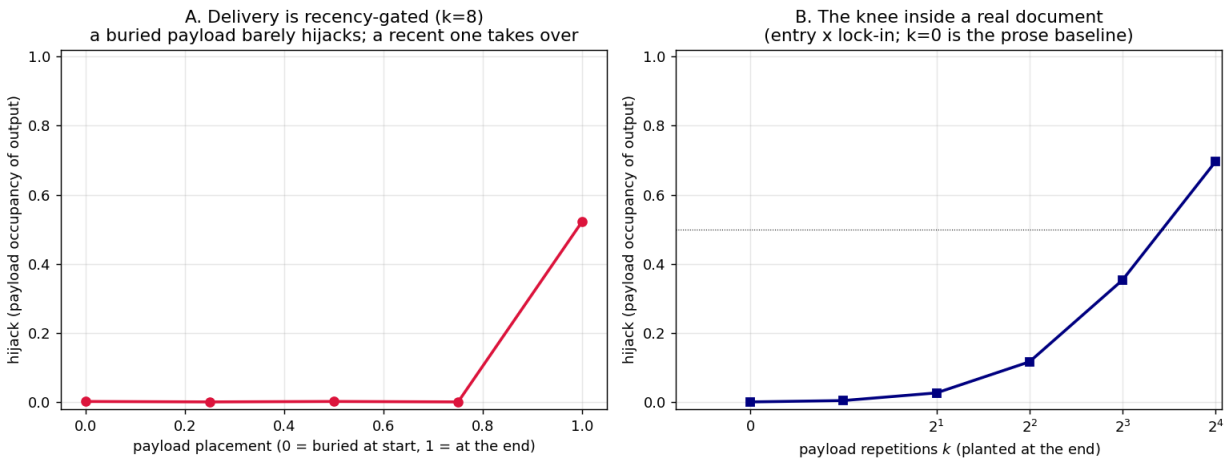


Figure 9: E5-lite. *Left:* hijack vs. placement—delivery is recency-gated, only a payload at the end takes over. *Right:* hijack vs. repetitions at the end—the knee inside a real document. Reproduced by `experiments/e5_hijack.py`.

when it sits at the very end (frac=1.0→0.52); buried anywhere earlier it is ≈ 0, even with just ∼16 tokens of prose after it—sharper than E1.5's filler gap, because coherent prose competes harder for the continuation. *Lock-in* (repetitions, at the end): a clean knee crossing $\frac{1}{2}$ around $k \approx 10$–12, *higher* than the bare-context knee (∼3): embedding in legitimate prose raises the poison cost.

C5 confirmed: hijack = entry × lock-in. Both must hold—the payload must be recent *and* repeated enough to overcome the legitimate content. The danger zone in a long context is the span near where generation happens; burying a payload, or strong surrounding context, suppresses the hijack.

13 E6: the context-window worm

Transmission (C6): an infected host's output (containing the payload) becomes the next host's context, passed as *text* (decoded then re-tokenized, as agents and RAG pipelines pass it); we count payload copies in each host's output across hops.

A real worm with a critical length. In same-model serial passage (Fig. 10, left), a length-1 string *propagates indefinitely* at an endemic load (∼12 copies/host) while longer strings *die out*, faster as p grows (gone by hop 2 at p=8): the basic reproduction number falls with string length and crosses 1 around $p \approx 1$–2, exactly the toy's prediction (short = epidemic, long = one-shot). The epidemic is carried by *super-spreader* payloads that fully take over a host's output (occupancy → 1, ending mid-payload so the next host re-enters); most payloads reproduce weakly and die, tying the critical length to the recency limit of E1.5. **Cross-tokenizer firebreak** (Fig. 10, right): same-tokenizer transmission survives to hop 6, but switching hosts to Pythia-160M collapses it after one hop—re-tokenizing fragments the payload so induction cannot copy it cleanly. A heterogeneous-model pipeline is naturally worm-resistant.

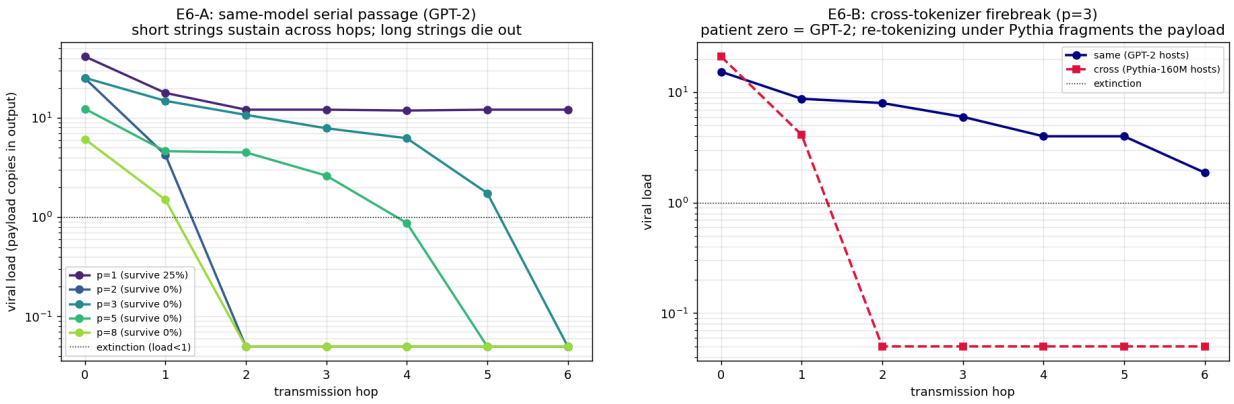


Figure 10: E6. *Left:* same-model serial passage—short strings (p=1) sustain at an endemic load while longer strings die out, faster as p grows (a critical length). *Right:* the cross-tokenizer firebreak—GPT-2→GPT-2 transmission persists, GPT-2→Pythia collapses after one hop. Reproduced by `experiments/e6_worm.py`.

14 The paired detector

The attacks of §11–13 all key on one quantity: the induction trust a repeated span earns. The defense watches the same quantity. We score a context by its mean *induction engagement*—the attention the canonical induction heads (from E2) pay to repeated-bigram continuations—which is high on a repeated/poisoned span and ≈ 0 on natural prose, regardless of whether the content is meaningful. As a binary detector (clean prose documents vs. the same documents with an OOD payload repeated k times), it separates the two *perfectly*: ROC-AUC = 1.0 at every $k \in \{2, 4, 8, 16\}$, with the score rising monotonically in k (Fig. 11). It fires already at k=2, well below the in-document hijack knee (∼10–12, §12): it flags the poisonable configuration *before* it is dangerous, and on the trust signal independent of usefulness.

Flagging—or capping the trust earned by—repeated spans that approach the condensation knee is precisely what neutralizes the E5 hijack and the E6 worm; it is the transformer realization of the toy's defensive corollary, *bound trust, not usefulness*. *Caveat:* the detector flags *any* induction-engaging repeated span, malicious or benign (boilerplate, templates, repeated instructions). That is the correct behavior under the thesis—such spans are genuinely poisonable (§11)—but it identifies the poisonable *configuration*, not malicious *intent*, so a deployment needs a benign-repetition allowlist or a trust cap rather than a hard block.

What the AUC above does not show. The negative class had *three* documents, so "AUC 1.0" is three-times-twelve pairwise comparisons, and no model-free baseline was run. A count of repeated n-grams needs no model and is expected to separate the same two classes just as well on this data; if it does, the induction score is an *explanation* of why repetition is dangerous, not a better alarm. The experiment has been rewritten to score 32 clean paragraphs, an n-gram baseline, and eight legitimately repetitive texts (a license header, code, a numbered list, a chorus, a CSV dump) for a false-positive rate at the zero-false-positive threshold, with bootstrap intervals; the rerun will replace Fig. 11.

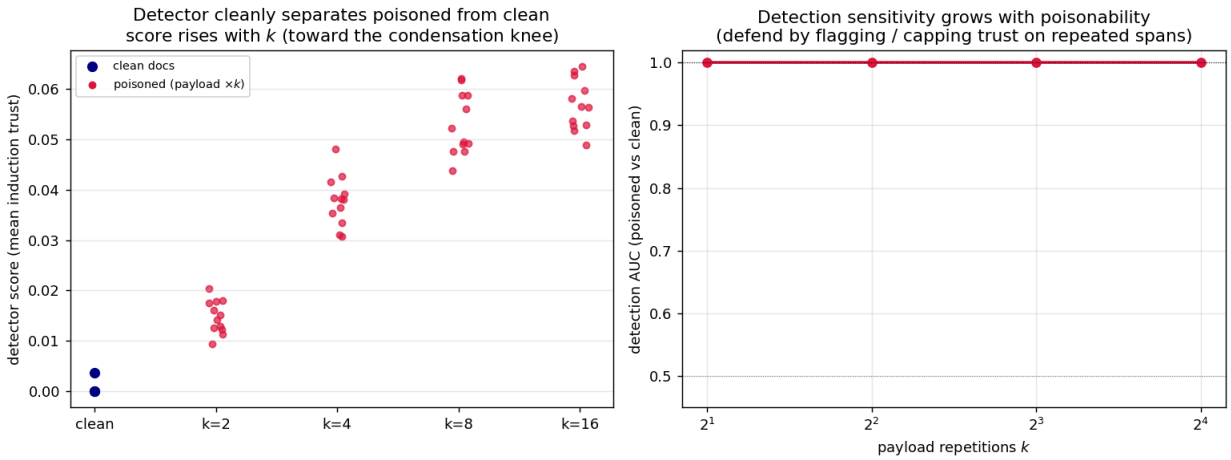


Figure 11: The detector. *Left:* score (mean induction trust) for clean docs vs. docs with a payload repeated k times—clean sits at ≈ 0, poisoned rises with k. *Right:* detection AUC vs. k ($= 1.0$ throughout). Reproduced by `experiments/e7_detector.py`.

15 E8 capstone: coherent self-extraction on instruct models

E8-lite established that in-context substitution is induction-carried but sharply depth-limited on GPT-2-small (a 2-step cascade succeeds only 0.19, below f^2, from cross-layer interference), predicting that a *coherent* self-extracting payload—and the "decode this, then follow it" self-execution loop—would need a capable, instruction-tuned model. We test both on three chat models, *strictly benignly*: the decoded payload is a harmless English sentence and the executed instruction is a harmless marker task ("reply with ACORN"); there is no harmful content by design—the aim is to characterize the mechanism and its detection surface, not to build an attack. **E8a** gives the model d nested substitution tables and an encoded message, scoring the LCS fidelity of its decode to plain English; **E8b** asks it to decode a benign instruction *and act on it*, scoring the self-execution rate; `surface-leak` records whether the benign marker is visible in the *encoded* form.

instruct model	E8a unfold (d=1/2/3)	E8b self-exec (d=1/2/3)	surface-leak
Qwen2.5-7B-Instruct	1.00 / 0.90 / 0.94	**1.00 / 1.00 / 1.00**	0.00
Llama-3.2-3B-Instruct	0.91 / 0.82 / 0.69	1.00 / **0.00 / 0.00**	0.00
gemma-2-2b-it	0.87 / 0.95 / 0.87	1.00 / 0.88 / 0.50	0.00

Table 9: E8 capstone (benign payloads). E8a: LCS fidelity of unfolding d-layer nested substitution into coherent English (toy 2-step limit 0.19). E8b: rate of decoding a benign instruction *and* executing it. surface-leak: fraction of encoded payloads in which the marker is visible pre-decode (0—the surface is gibberish).

The capability dependency holds: coherent self-extraction needs a capable model, and capable models have it. All three instruct models shatter the toy's 0.19 limit on unfolding (E8a 0.69–1.00)—a capable model decodes even three nested substitution layers into coherent English, exactly what the induction-only toy could not. **Self-execution is the harder, capability-gated step, and it dissociates from decoding.** Decode-*and*-act (E8b) is strictly harder than decode (E8a): Qwen2.5-7B self-executes perfectly through depth 3 (1.00), gemma-2-2B degrades gracefully

(1.00→0.88→0.50), and Llama-3.2-3B falls off a cliff—1.00 at depth 1 then 0.00 at depth ≥2 while it is still *decoding* at 0.82. Acting on one's own in-context decoding is a compositional step more demanding than either half alone, and only the 7B does it robustly with depth. **The evasion surface is clean, but its mitigation is structural.** `surface-leak`=0 everywhere: a content filter sees only gibberish because the payload materializes only after decoding. The defense is therefore not content inspection but *structure*—the substitution-table and "decode and follow" scaffold is a conspicuous, induction-engaging repeated-pair block, exactly the signal the paired detector (§14) keys on, independent of the hidden payload. (Caveats: small n, single seed, greedy; benign markers by design.)

16 At scale: the downstream claims C4/C5/C6

The three downstream claims had lite/proxy results on GPT-2-small; we now test them in their real settings on instruct models (Qwen2.5-7B-Instruct unless noted). The outcomes are *mixed*— one confirmation, one bound, one that is *worse* than the toy predicted— which is exactly the pre-registered discipline: either direction is informative.

C4 with a real task (§ E4-scale): design flaw found, rerun pending. E4-lite used a naturalness *proxy* for usefulness. Here usefulness is a genuine task: benign *synthetic* facts the model cannot already know, so a context containing the fact is the only source of the answer. The 2026-07-02 run reported $\mathrm{corr}(P,U)$=+0.09 and $\mathrm{corr}(P,T)$=+0.71 over six facts. Two problems invalidate it as a test of C4. First, both "non-repetitive" cells were the *same* filler paragraph written twice, so the 2×2 contrasted eight-fold with two-fold repetition. Second, poisonability was the copy-rate of a raw continuation of "span ×8", which is ≈ 1 for any repeated span by construction—it measures that the context *is* a loop, not that the context makes a planted span more likely to be reproduced. The script has been rewritten: non-repetitive cells are built from distinct sentences and length-matched in tokens; poisonability is reported both as copy-rate and as reproduction of a *planted* probe span (the plan's original measure); 32 facts; bootstrap intervals over facts. Until it is rerun, C4 stands on the GPT-2-small proxy (§11) only.

C5 is bounded: a task query defuses the raw-copy hijack (§ E5-scale). E5-lite hijacked a base model's *continuation* with a recent, repeated payload. At scale we plant an OOD payload $\times k$ inside a ∼4000-token benign retrieval dump on an instruct model and ask it to *answer a query*: hijack (payload occupancy of the answer) is 0.00 at every placement and for k up to 32. The same model reproduces a planted span in raw continuation (E1.4), so this is not an inability to copy—the *task query anchors generation away from the payload*. The raw-repetition hijack is thus a base-model-continuation phenomenon; a capable instruct model doing a real task is not hijacked by it. A well-posed query is itself a mitigation; the residual RAG threat is *semantic* injection, outside this project's benign-OOD copy-dynamics scope.

C6, re-analyzed: no reproduction number above one; the firebreak depends on the run (§ E6-scale). Serial passage on larger/instruct models: GPT-2-small, Pythia-2.8B and Llama-3.2-3B lose the payload within six hops for every $p \geq 2$, while Qwen2.5-7B (base and instruct) still carries it at hop six for p=2–8 in 20–60% of five trials. The first draft called this "the worm gets worse at scale." Computing the per-hop multiplication from the committed mean trajectories (no per-trial data were stored) gives a different picture: *no* cell has a ratio above one. The Qwen chains that survive have a *flat* mean load across hops ($29, 29, 29, 29$ at p=2; $8, 8, 8, 8$ at p=5), which

is the signature of a host emitting its full T=96 tokens as payload every hop—a locked repetition loop [8] whose load is fixed by T/p, not an epidemic with a reproduction number. Unlocked chains decay by 0.5–0.7× per hop until extinction. The toy's $R_0 \approx T/(p\,k_{\text{thresh}})$ is therefore not what these data show; what varies with model and p is the per-trial *probability of locking*, and a 7B model locks more readily than a 3B one. That is a claim about degeneration propensity, testable with the per-trial trajectories the scripts now store.

Two cross-tokenizer firebreak matrices (5 patient-zero families × 5 host families) were run on 2026-07-02 and only the second was reported. Under an exact permutation test (null: the host's tokenizer is irrelevant, so columns are exchangeable within a row; 5^5 assignments): at the script defaults—3 hops, 16 seed copies, 5 trials—the diagonal mean is 4.0 vs. 2.4 off-diagonal, the diagonal is the row maximum in 2 of 5 rows and is *zero* in two, and p=0.24: no detectable same-tokenizer advantage. In the reported re-run—1 hop, 32 seed copies, 8 trials—the diagonal mean is 14.2 vs. 8.8, row maximum in 3 of 5, p=0.004. So under heavy seeding a single re-tokenization attenuates transmission by about 1.6×; over three hops on 0.5–2B hosts, transmission is too weak in *both* conditions to show a firebreak. The protocol change between the runs is now recorded; the queued higher-n sweep over seeding strength at three hops is what decides between the two readings.

17 Threats to validity

Beyond the small samples ($R\sim$3–6 payloads per cell on GPT-2-small, 3 payloads × 8 trials per cell at scale, 4–8 trials per cell in Track C, single seeds), the standing concerns are: **grid floor**—for $p \geq 3$ the interpolated knee lies in the first grid interval in three quarters of cells, so "N^*" is a one-shot rate and cannot show sharpness or a scale trend (§9); **raw data**—the Track A and most Track B result files were never committed, so their tables are transcriptions and cannot be re-analyzed (the scripts now snapshot to a tracked directory); **tokenization**—length is in tokens and OOD spans tokenize unstably, addressed by a round-trip-stability filter; **decoding**—the knee persists under *greedy* decoding (with lower N^*, less sampling noise to break reproduction), so it is a model property, not a sampling artifact; a full temperature sweep is still future work; **instruction-tuned models** may comment or refuse rather than copy, so base models are used for the clean mechanism and chat models are a separate condition—now run (E1.4, §9), where the knee persists with only a modest threshold rise; **context-length vs.** N—addressed by E1.5 (§7): the knee persists at fixed total length, so it is an N effect, though reproduction is distance-sensitive; and **scale**—the behavioral knee extends cleanly to 7B across four families and is flat with size (§9), and the *mechanistic* attribution now holds there too (E2 to 7B in three families, §10.1); what remains is thin-n: the Track C reversals (§16) rest on ∼4–8 payloads/facts per cell and warrant higher-power replication.

18 What is established, and what is not

On GPT-2-small, with controls and confidence intervals: a repeated out-of-distribution string is regenerated past a sharp condensation knee (C1); the knee *decreases* with string length, the induction-like length-robustness predicted by the toy (C3); the knee is a repetition effect (not context length) though distance-sensitive (E1.5); reproduction is a cue-triggered copy rather than a frequency bias (C2, behaviorally); and—the new result— ablating the canonical GPT-2 induction heads *causally* collapses reproduction near the knee, far beyond random-head damage, so **C2 is mechanistically established** on this model (E2). The downstream claims also hold on GPT-2-small: poisonability is dissociated from usefulness and aligned with induction trust (C4, §11); a planted span hijacks generation as entry × lock-in (C5, §12); and the worm has a critical string

length and a cross-tokenizer firebreak (C6, §13). The phenomena replicate in a second model family (Pythia-160M). So all six conjectures C1–C6 have a qualitative real-transformer result, and a paired detector (§14) realizes the toy's bound-trust defense (poisoned vs. clean documents separate at AUC 1.0 on the induction-trust signal). At scale, what holds is narrower than the first draft said. The one-shot copy rate for multi-token spans is $\geq \frac{1}{2}$ in 36 of 48 cells from 355M to 7B across four families (§9); the "knee" for those payloads is the grid floor and its flatness with parameter count is not a finding. Copying persists in instruction-tuned models, which need more occurrences but still copy (E1.4). Ablating the top $\sim 5\%$ of heads by induction score collapses reproduction across scale and three families (124M–7B; GPT-2, Pythia, Qwen2.5), so **C2 is mechanistically established at scale** (§10.1); the scale-matched ablation, and the false null a fixed head count produced, is this report's clearest methodological contribution. The self-extraction capstone (E8) succeeds on instruct models. One claim is *bounded*: the raw-repetition RAG hijack (C5) does *not* transfer to a capable instruct model answering a real query—the task query defuses it, so the residual RAG threat is semantic rather than copy-dynamic. Two at-scale results are *withdrawn pending rerun*: the C4 test with a real task (repeating "non-repetitive" cells, a copy-rate that is ≈ 1 by construction) and the detector's AUC (three negatives, no baseline). And the worm (C6) has been *re-read*: no cell shows a reproduction number above one; the 7B "endemic" chains are locked repetition loops, and the tokenizer firebreak is significant in a one-hop run and undetectable in the three-hop run that was not reported. What *remains* is the higher-powered replication those reruns provide, the very-long-context (128k) and larger host-population regimes, and the *semantic*-injection version of the RAG threat, which is outside this project's benign copy-dynamics scope.

References

[1] D. Shumow. *Trust Is the Attack Surface: A Cryptanalytic Reading of In-Context Memory Poisoning in a Markov Mixture Model.* Project `trust-is-the-attack-surface`, 2026.

[2] D. Shumow. *Contagious Context: Self-Reproducing and Transmissible Token Strings in a Markov Cache.* Project `trust-is-the-attack-surface`, 2026.

[3] C. Olsson et al. *In-context Learning and Induction Heads.* Transformer Circuits, 2022.

[4] A. Bietti, V. Cabannes, D. Bouchacourt, H. Jégou, L. Bottou. *Birth of a Transformer: A Memory Viewpoint.* NeurIPS, 2023.

[5] S. Cohen, R. Bitton, B. Nassi. *Here Comes the AI Worm: Unleashing Zero-click Worms that Target GenAI-Powered Applications.* 2024.

[6] K. Greshake et al. *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.* AISec, 2023.

[7] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever. *Language Models are Unsupervised Multitask Learners.* OpenAI, 2019.

[8] A. Holtzman, J. Buys, L. Du, M. Forbes, Y. Choi. *The Curious Case of Neural Text Degeneration.* ICLR, 2020 (arXiv 2019).

[9] J. Xu, X. Liu, J. Yan, D. Cai, H. Li, J. Li. *Learning to Break the Loop: Analyzing and Mitigating Repetitions for Neural Text Generation.* NeurIPS, 2022.

[10] C. McDougall, A. Conmy, C. Rushing, T. McGrath, N. Nanda. *Copy Suppression: Comprehensively Understanding an Attention Head.* 2023.

[11] S. Jelassi, D. Brandfonbrener, S. M. Kakade, E. Malach. *Repeat After Me: Transformers are Better than State Space Models at Copying.* ICML, 2024.

[12] C. Anil et al. *Many-Shot Jailbreaking.* NeurIPS, 2024.

[13] M. Nasr, N. Carlini, J. Hayase, M. Jagielski, A. F. Cooper, D. Ippolito, C. A. Choquette-Choo, E. Wallace, F. Tramèr, K. Lee. *Scalable Extraction of Training Data from (Production) Language Models.* 2023.

[14] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang. *Lost in the Middle: How Language Models Use Long Contexts.* TACL, 2024 (arXiv 2023).